



ESCUELA DE  
INGENIERÍA EN CIENCIAS Y SISTEMAS  
FACULTAD DE INGENIERÍA  
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



|                 |                    |
|-----------------|--------------------|
| Día, Fecha:     | JUEVES, 05/02/2026 |
| Hora de inicio: | 17:20              |

# ANÁLISIS Y DISEÑO DE SISTEMAS 2 SECCIÓN B

Escuela de Ingeniería en Ciencias y Sistemas

NOMBRE DEL TUTOR



Universidad de San Carlos de Guatemala  
Facultad de Ingeniería  
Escuela de Ingeniería de Ciencias y Sistemas



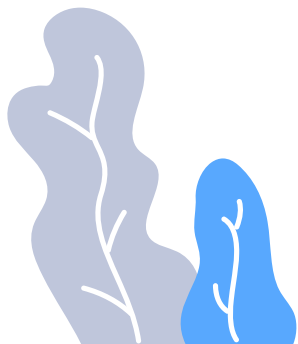
# Patrones de Diseño

## Análisis y Diseño de Sistemas 2

Inicio 17:20

Jueves dd/MM/YYYY

---



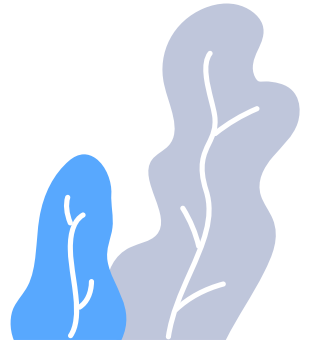
# Agenda

- 01 Patrones de diseño
- 02 SOLID
- 03 Patrones Creacionales
- 04 Foro de la semana
- 05 Dudas y comentarios



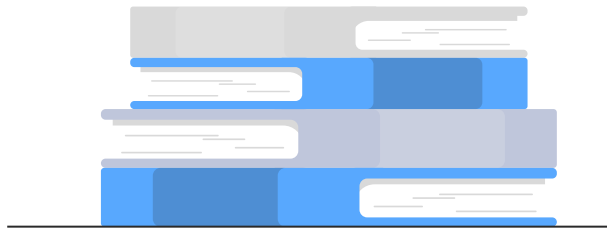
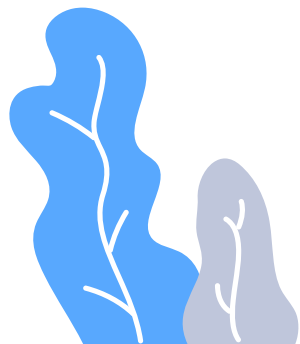
# Competencia Que Desarrollaremos

Diseñar y aplicar patrones de diseño de software utilizando principios de diseño y catálogos de estilos garantizando la calidad estructural y de comportamiento del sistema.



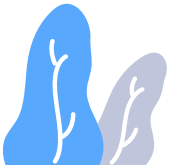
01

# Patrones de Diseño



# ¿Qué es un patrón?

Un "patrón" puede referirse a varias cosas, pero generalmente implica un modelo, regla o estructura repetitiva que se observa en objetos, eventos o datos. En términos generales, un patrón es una regularidad o disposición predecible que se puede identificar y utilizar para comprender o predecir fenómenos.



# ¿Qué es un patrón de diseño?

“Un patrón de diseño es una solución comprobada a un problema común en un contexto específico. No es código, sino una guía para resolver un problema de diseño que se repite en el desarrollo de software”.

**Head First Design Patterns**

**Autores:** Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra

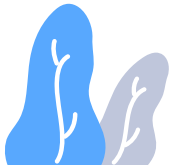
**Año:** 2004

“Un patrón de diseño es una descripción de objetos y clases que se comunican y están personalizados para resolver un problema general de diseño en un contexto específico”.

**Design Patterns: Elements of Reusable Object-Oriented Software**

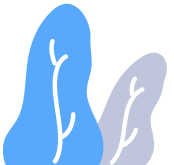
**Autores:** Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (Gang of Four – GoF)

**Año:** 1994



Un patrón de diseño es una solución reutilizable y comprobada para resolver un problema común que ocurre durante el diseño de software orientado a objetos.

No es una función ni una librería: es una plantilla de solución que se puede adaptar al código según el contexto.





# Importancia de los Patrones de Diseño



## **Reutilización de soluciones:**

Permiten resolver problemas comunes con soluciones ya probadas y eficientes.

## **Mejora la comprensión**

Facilitan la comunicación entre desarrolladores usando un lenguaje común.

## **Diseño limpio y estructurado**

Promueven la separación de responsabilidades y un bajo acoplamiento.

## **Mantenibilidad**

El código es más modular, legible y fácil de modificar.

## **Escalabilidad**

Facilitan agregar nuevas funcionalidades sin romper el sistema.

Los patrones representan un conocimiento colectivo de la comunidad experta.

# Formato Estandar Para Documentar

| Campo         | ¿Qué contiene?                       |
|---------------|--------------------------------------|
| Nombre        | Nombre del patrón                    |
| Problema      | Qué problema resuelve                |
| Participantes | Clases u objetos involucrados        |
| Solución      | Cómo lo resuelve (breve descripción) |
| Diagrama UML  | Relación entre clases                |
| Consecuencias | Ventajas y desventajas               |

| Campo         | ¿Qué contiene?                                                                                                                                                                                                                                                                                                      |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nombre        | Observer                                                                                                                                                                                                                                                                                                            |
| Problema      | Se necesita que varios objetos se actualicen automáticamente cuando otro objeto cambia su estado.                                                                                                                                                                                                                   |
| Participantes | <ul style="list-style-type: none"><li>- Subject (Sujeto): el objeto observado</li><li>• Observer (Observador): interfaz que define la actualización</li><li>• ConcreteObserver: implementaciones que reaccionan a los cambios</li><li>• ConcreteSubject: mantiene el estado y notifica a los observadores</li></ul> |



# TIPOS DE PATRONES DE DISEÑO

## PATRONES DE DISEÑO

### DE CREACION

Abstrac factory  
Factory Method  
Builder  
Prototype  
Singleton

### De estructura

Adater  
compositer  
Decorator  
Facade  
Proxy  
Bridge

### De comportamiento

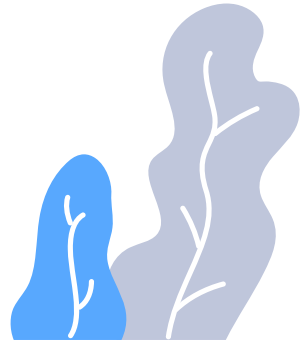
Command  
Interpreter  
Iterator  
Mediator  
Observer  
Strategy  
Template method

# Principios de Diseño en Software Orientado a Objetos

Los principios de diseño son reglas o buenas prácticas que ayudan a construir software modular, mantenible, reutilizable y fácil de entender.  
Son la base para aplicar correctamente los patrones de diseño.

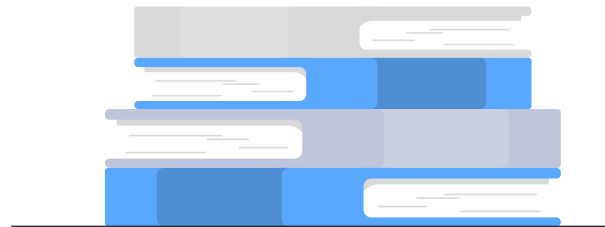
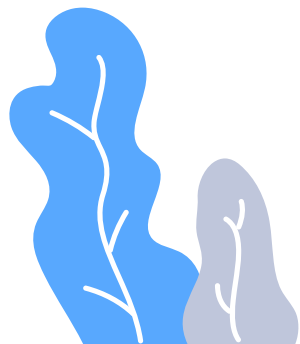
## ¿Por qué son importantes?

1. Permiten organizar el código de forma coherente y flexible.
2. Reducen la complejidad del software
3. Facilitan la colaboración en equipos grandes



02

SOLID



# SRP: THE SINGLE RESPONSIBILITY PRINCIPLE

## Human resources

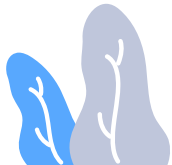
The project team is responsible for the successful execution of the project. Our team is composed of experienced professionals with the necessary skills and expertise to complete the project on time and within budget

## Financial resources

Our project budget is \$100,000. This budget includes all expenses associated with the project, including salaries and equipment. We have allocated these resources to ensure that we are able to complete the project within budget

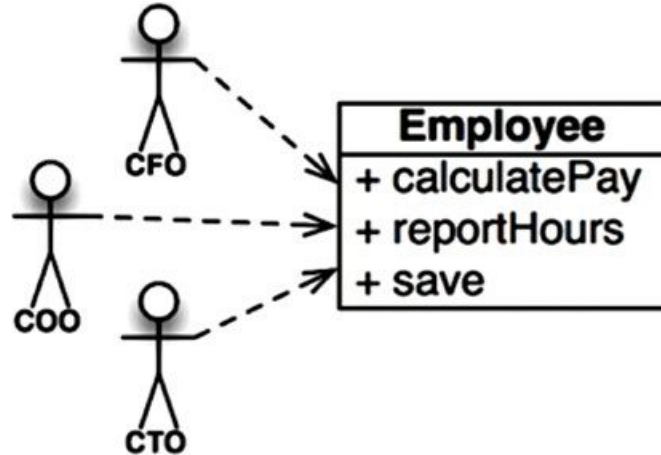
## Physical resources

Our project requires a number of specialized pieces of equipment. We will be using [insert equipment and their functions]. All equipment is in good working condition and has been tested and calibrated prior to use



# SRP: THE SINGLE RESPONSIBILITY PRINCIPLE

El Principio de Responsabilidad Única (SRP) establece que una clase debe tener una sola razón para cambiar, es decir, debe encargarse de una única responsabilidad o función dentro del sistema.

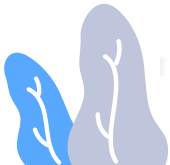


# Viola SRP (Single Responsibility Principle)

```
class Employee:
    def calculate_pay(self):
        print("Calculando salario")

    def report_hours(self):
        print("Reportando horas")

    def save(self):
        print("Guardando empleado en BD")
```





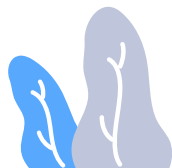
# Solucion

```
class Employee:
    def __init__(self, name, hours, rate):
        self.name = name
        self.hours = hours
        self.rate = rate

class PayrollService:
    def calculate_pay(self, employee):
        return employee.hours * employee.rate

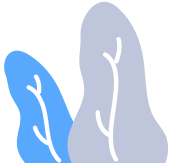
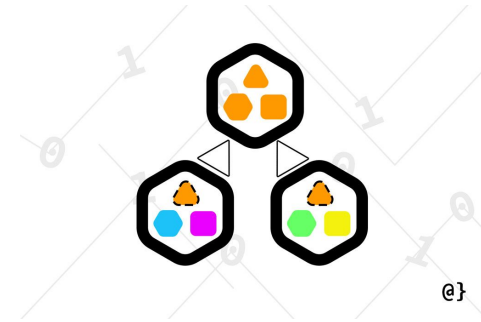
class HoursReporter:
    def report_hours(self, employee):
        print(f"{employee.name} trabajó {employee.hours} horas")

class EmployeeRepository:
    def save(self, employee):
        print(f"Empleado {employee.name} guardado en base de datos")
```



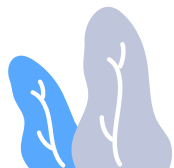
# OCP: THE OPEN-CLOSED PRINCIPLE

El Principio Abierto/Cerrado (OCP) establece que las entidades de software (clases, módulos, funciones) deben estar abiertas para extensión, pero cerradas para modificación.



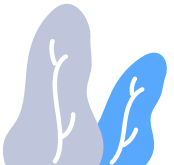
# Rompe OCP(THE OPEN-CLOSED PRINCIPLE)

```
class SalaryCalculator {  
    double calculate(String type, int hours, double rate) {  
        if (type.equals("FULL_TIME")) return hours * rate;  
        if (type.equals("PART_TIME")) return hours * rate * 0.8;  
        if (type.equals("INTERN")) return hours * rate * 0.5;  
        throw new IllegalArgumentException("Unknown type");  
    }  
}
```



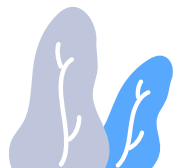
# Solucion

```
interface Employee {  
    double calculatePay();  
}  
  
class FullTimeEmployee implements Employee {  
    private final int hours;  
    private final double rate;  
  
    FullTimeEmployee(int hours, double rate) {  
        this.hours = hours;  
        this.rate = rate;  
    }  
  
    public double calculatePay()  
        return hours * rate;  
}
```

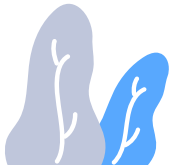


```
class PartTimeEmployee implements Employee {  
    private final int hours;  
    private final double rate;  
  
    PartTimeEmployee(int hours, double rate) {  
        this.hours = hours;  
        this.rate = rate;  
    }  
  
    public double calculatePay()  
        return hours * rate * 0.8;  
}
```

```
class Intern implements Employee {  
    private final int hours;  
    private final double rate;  
  
    Intern(int hours, double rate) {  
        this.hours = hours;  
        this.rate = rate;  
    }  
  
    public double calculatePay() {  
        return hours * rate * 0.5;  
    }  
}
```



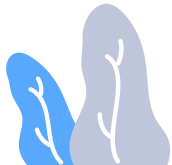
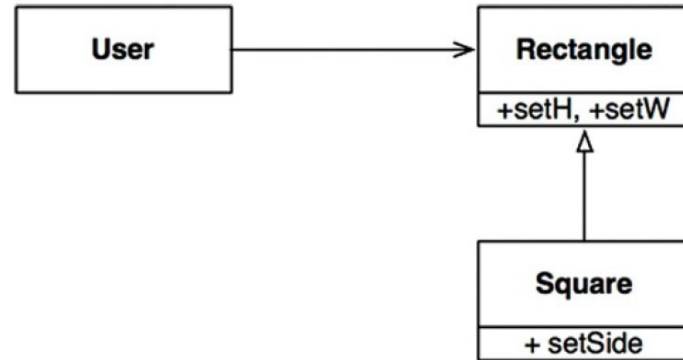
```
public class Main {  
    public static void main(String[] args) {  
        Employee[] employees = {  
            new FullTimeEmployee(40, 15),  
            new PartTimeEmployee(20, 15),  
            new Intern(30, 10)  
        };  
  
        for (Employee e : employees) {  
            System.out.println(e.calculatePay());  
        }  
    }  
}
```



# LSP: Liskov Substitution Principle

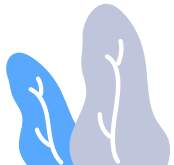
El Principio de Sustitución de Liskov establece que los objetos de una clase derivada debe poder sustituir a los objetos de su clase base sin alterar el comportamiento correcto del programa.

Si una clase hereda de otra, debería poder usarse en su lugar sin que el programa falle o se comporte de forma inesperada.



# ISP: THE INTERFACE SEGREGATION PRINCIPLE

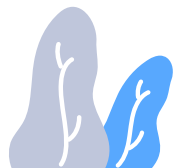
El Principio de Segregación de Interfaces (ISP) establece que una clase no debe verse obligada a depender ni a implementar métodos que no utiliza. Para ello, es preferible diseñar interfaces pequeñas y específicas, en lugar de interfaces grandes y generales, de modo que cada cliente dependa únicamente de las funcionalidades que realmente necesita





# Rompe ISP

```
interface Printer {  
    void print();  
    void scan();  
    void fax();  
}  
  
class SimplePrinter implements Printer {  
  
    public void print() {  
        System.out.println("Imprimiendo...");  
    }  
  
    public void scan() {  
        throw new UnsupportedOperationException("No escanea");  
    }  
  
    public void fax() {  
        throw new UnsupportedOperationException("No envía fax");  
    }  
}
```



# Solucion

```
interface Printable {  
    void print();  
}
```

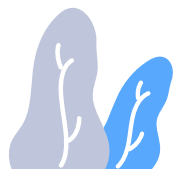
```
interface Scannable {  
    void scan();  
}
```

```
interface Faxable {  
    void fax();  
}
```



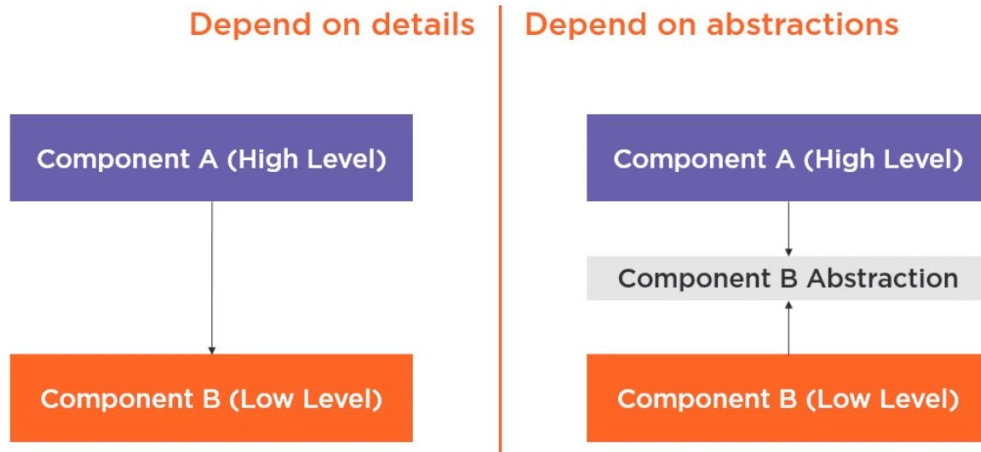
```
class SimplePrinter implements Printable {  
    public void print() {  
        System.out.println("Imprimiendo...");  
    }  
}
```

```
class MultiFunctionPrinter implements Printable, Scannable, Faxable {  
    public void print() {  
        System.out.println("Imprimiendo...");  
    }  
  
    public void scan() {  
        System.out.println("Escaneando...");  
    }  
  
    public void fax() {  
        System.out.println("Enviando fax...");  
    }  
}
```

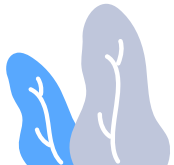
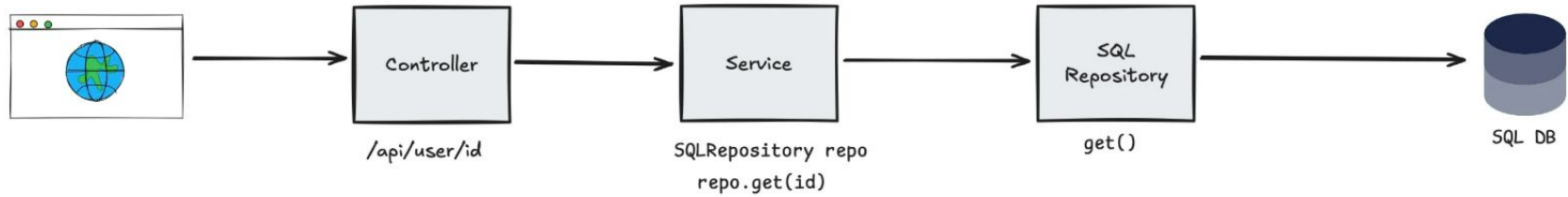


# DIP: THE DEPENDENCY INVERSION PRINCIPLE

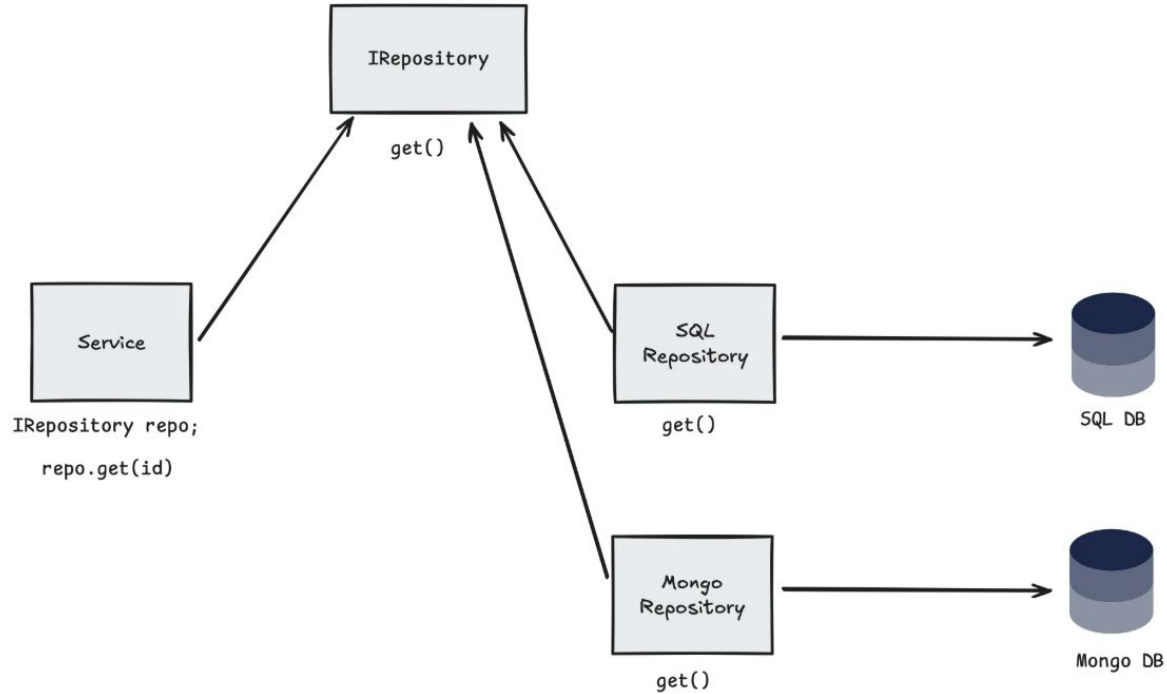
El Principio de Inversión de Dependencias (DIP) establece que los módulos de alto nivel no deben depender de módulos de bajo nivel, sino que ambos deben depender de abstracciones. Además, las abstracciones no deben depender de los detalles, sino que los detalles deben depender de las abstracciones.



# Sin DIP

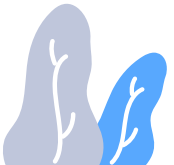


# Con DIP



# Código

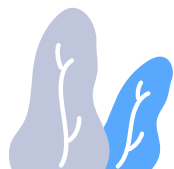
```
public class UserService {  
    SQLRepository repository = new SQLRepository();  
  
    public void get(String id) {  
        repository.get(id);  
    }  
}  
  
public class SQLRepository {  
    public void get(String id) {  
        System.out.println("User is fetched from SQL DB");  
    }  
}
```



```
public interface IRepository {  
    void get(String id);  
}
```

```
public class SQLRepository implements IRepository {  
    @Override  
    public void get(String id) {  
        System.out.println("User is fetched from SQL DB");  
    }  
}
```

```
public class MongoRepository implements IRepository {  
    @Override  
    public void get(String id) {  
        System.out.println("User is fetched from Mongo DB");  
    }  
}
```





```
public class UserService {
    IRepository repository;

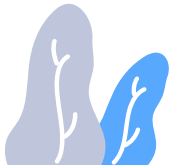
    public UserService(IRepository repository) {
        this.repository = repository;
    }

    public void get(String id) {
        repository.get(id);
    }

    public void setRepository(IRepository repository) {
        this.repository = repository;
    }
}

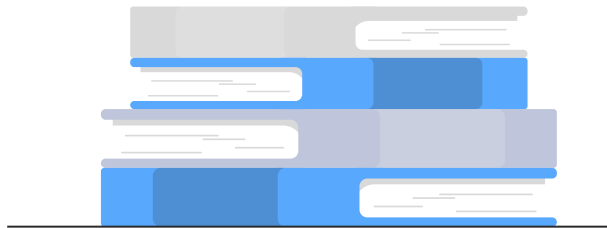
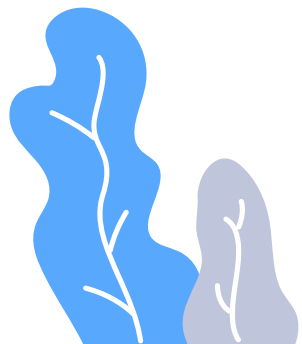
public class Main {
    public static void main(String[] args) {
        String id = "xyz";
        UserService userService = new UserService(new SQLRepository());
        userService.get(id);

        userService.setRepository(new MongoRepository());
        userService.get(id);
    }
}
```



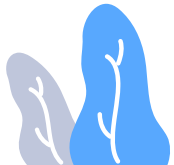
03

# Patrones Creacionales



Son un conjunto de soluciones probadas y reutilizables en el diseño de software, específicamente enfocadas en cómo se crean los objetos. Su objetivo principal es abstraer el proceso de instanciación de objetos, es decir, separar la forma en que los objetos son contruidos de su representación final y de las clases concretas que los implementan.

- Hacer que tu sistema sea independiente de cómo se crean sus objetos. Esto significa que puedes cambiar la forma en que se instancian los objetos sin afectar el código que los utiliza.
- Incrementar la flexibilidad y la reutilización del código. Al encapsular la lógica de creación, puedes crear diferentes tipos de objetos o configuraciones de objetos de manera más sencilla.
- Reducir el acoplamiento entre las clases, lo que hace que tu código sea más fácil de mantener y extender.
- Controlar el proceso de creación de objetos, cómo asegurar que una clase sólo tenga una instancia (Singleton) o construir objetos complejos paso a paso (Builder).

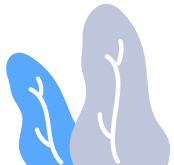


# Singleton

Singleton es un patrón de diseño creacional que asegura que una clase tenga una sola instancia en toda la aplicación y provee un punto de acceso global a esa instancia.

El patrón Singleton se usa cuando una clase deba tener una instancia disponible para todos los clientes; por ejemplo, un único objeto de base de datos compartido por distintas partes del programa.

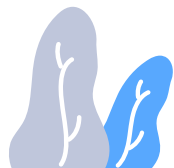
El patrón Singleton deshabilita el resto de las maneras de crear objetos de una clase, excepto el método especial de creación. Este método crea un nuevo objeto, o bien devuelve uno existente si ya ha sido creado.



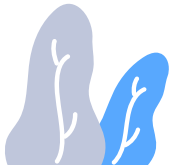
```
class Logger:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            print("New Instance")
        return cls._instance

    def log(self, message):
        print(f"[LOG]: {message}")
```



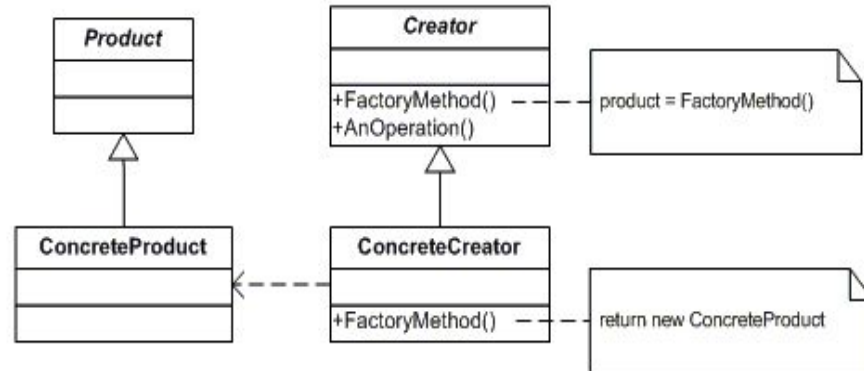
```
public class Logger {  
  
    private static Logger instance;  
  
    private Logger() { }  
  
    public static Logger getInstance() {  
        if (instance == null) {  
            instance = new Logger();  
        }  
        return instance;  
    }  
  
    public void log(String message) {  
        System.out.println("[LOG]: " + message);  
    }  
}
```



# Factory Method

Factory Method es un patrón de diseño creacional que se utiliza para encapsular la creación de objetos, evitando que el código cliente dependa directamente de clases concretas.

En lugar de crear objetos directamente, la creación se delega a un método especializado, permitiendo mayor flexibilidad en el sistema.



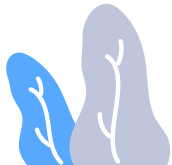
# ¿Qué problema resuelve?

Factory Method se utiliza cuando:

- El tipo de objeto a crear puede variar
- Existen múltiples implementaciones posibles
- Se quiere evitar el uso de **new** en muchos lugares
- Se busca reducir acoplamiento

Sin este patrón, el código suele llenarse de:

- condicionales (if / switch)
- dependencias rígidas
- dificultades para extender el sistema





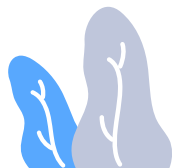
# Beneficios y relación con SOLID

## Beneficios

- Reduce el acoplamiento
- Facilita la extensión del sistema
- Centraliza la lógica de creación
- Mejora el mantenimiento del código

## Relación con SOLID

- **OCP**: se pueden agregar nuevos tipos sin modificar código existente
- **DIP**: el código depende de abstracciones, no de implementaciones
- **SRP**: la creación de objetos se separa del uso



```
class Notification(ABC):
```

```
    @abstractmethod
```

```
    def send(self):
```

```
        pass
```

```
class EmailNotification(Notification):
```

```
    def send(self):
```

```
        print("Enviando email")
```

```
class SMSNotification(Notification):
```

```
    def send(self):
```

```
        print("Enviando SMS")
```

```
class NotificationFactory(ABC):
```

```
    @abstractmethod
```

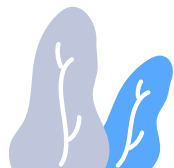
```
    def create_notification(self):
```

```
        pass
```

```
class EmailFactory(NotificationFactory):
```

```
    def create_notification(self):
```

```
        return EmailNotification()
```



# PATRONES DE DISEÑO



# Ejemplo Práctico

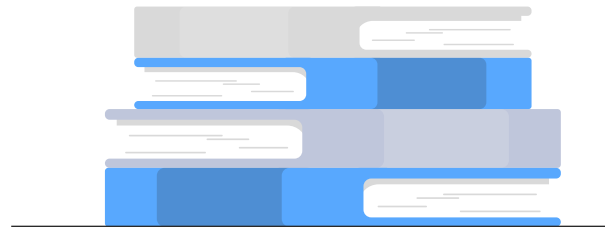
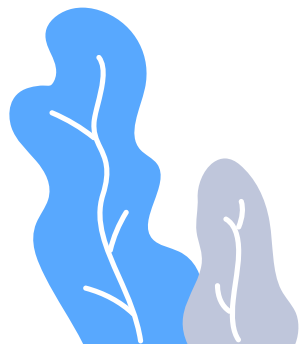


# Actividad en Clase



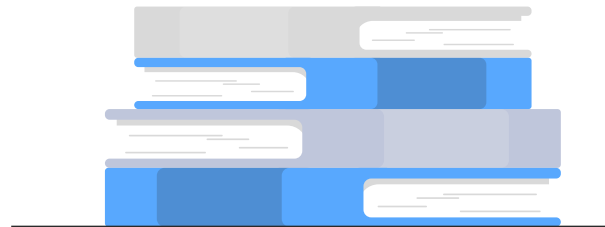
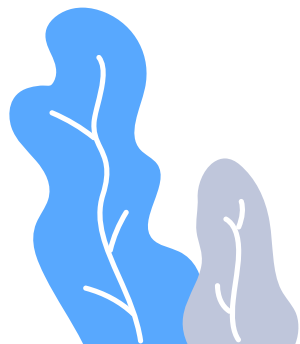
# 04

## Foro de la semana



05

Dudas



¡GRACIAS POR SU  
ATENCIÓN!

